

GUÍA DE LABORATORIO N°7

Planificación de Procesos y Programación Concurrente

Docente: Camilo Fuentes Beals. PhD.

1. Objetivos del Laboratorio

Al finalizar la guía, el estudiante será capaz de:

- Calcular métricas de planificación para políticas apropiativas y no apropiativas
- Implementar y analizar Race Conditions en código Python
- Aplicar mecanismos de sincronización con semáforos y mutex
- Comparar el impacto de distintas políticas sobre el rendimiento del sistema.

2. Antecedentes teóricos

Esta guía integra los contenidos de la Semana 9 del curso: políticas de planificación de CPU y principios de programación concurrente. Antes de iniciar las actividades, asegúrese de dominar los siguientes conceptos revisados en clase:

- Round Robin (RR) con quantum q y su impacto en el tiempo de espera y throughput.
- Diferencia estructural entre planificación apropiativa (preemptive) y no apropiativa (non-preemptive).
- Criterios de evaluación de políticas: utilización de CPU, productividad, tiempo de retorno, espera y respuesta.
- Race condition como consecuencia de acceso no coordinado a recursos compartidos.
- Los tres requisitos de la sección crítica: mutua exclusión, progreso y espera acotada.
- Mecanismos de sincronización: Busy Waiting, Semáforos, Mutex y Monitores.

3. Actividades

Actividad 1 - Análisis de Políticas de Planificación

1.1 - Contexto

En esta actividad calculará métricas de rendimiento para tres políticas de planificación sobre el mismo conjunto de procesos. El objetivo es evidenciar cuantitativamente por qué la elección de política tiene impacto directo en la experiencia del usuario y en la eficiencia del sistema.

1.2 - Escenario

Considere el siguiente conjunto de procesos. Todos los tiempos están expresados en unidades de tiempo (ut). Asuma que el tiempo de cambio de contexto es despreciable para los cálculos de las partes A y B.

Proceso	Llegada (ut)	Ráfaga CPU (ut)	Prioridad	Notas
P1	0	8	3	Proceso largo inicial
P2	1	4	1	Alta prioridad
P3	2	9	4	Proceso más largo
P4	3	5	2	Prioridad media-alta
P5	4	2	5	Proceso corto

1.3 – FCFS

Construya el diagrama de Gantt para FCFS. Recuerde que en FCFS los procesos se atienden en orden de llegada y una vez que la CPU es asignada no se retira hasta que el proceso termina o se bloquea voluntariamente.

Diagrama de Gantt FCFS - Construya aquí:

Tabla de métricas FCFS — Complete:

Proceso	Llegada	Ráfaga	Inicio	Finalización	T. Espera
P1	0	8			
P2	1	4			
P3	2	9			
P4	3	5			
P5	4	2			
Promedio	—	—	—	—	—

1.4 – Round Robin (Apropiativo, $q = 3$)

Construya el diagrama de Gantt para Round Robin con quantum $q = 3$ ut. Los procesos llegan en los tiempos indicados en la tabla. Si un proceso llega exactamente cuando termina el quantum del proceso actual, el proceso entrante se añade al final de la cola antes de que el proceso actual regrese (si no terminó).

Diagrama de Gantt RR $q=3$ - Construya aquí:

Tabla de métricas RR $q=3$ — Complete:

Proceso	Llegada	Ráfaga	# Turnos RR	Finalización	T. Espera
P1	0	8			
P2	1	4			
P3	2	9			
P4	3	5			
P5	4	2			
Promedio	—	—	—	—	—

1.5 – Preguntas de Análisis Comparativo

- ¿Qué política produjo un menor tiempo de espera promedio en este escenario? ¿A qué factor o factores se atribuye esa diferencia?
- Cuente el número total de cambios de contexto generados por Round Robin. Si cada cambio de contexto tiene un costo de 0.5 ut, recalcule el tiempo de espera promedio real de RR incluyendo ese overhead.
- Identifique qué proceso sufre el mayor tiempo de espera en FCFS y explique por qué. Nombre el fenómeno involucrado y proponga una modificación al algoritmo para mitigarlo.
- ¿En qué tipo de sistema computacional elegiría FCFS y en cuál preferiría RR? Justifique su respuesta considerando el tipo de carga de trabajo

Actividad 2 – Race Condition y Sección Crítica

2.1 - Contexto

En esta actividad reproducirá una Race Condition en Python, observará su efecto de forma experimental, identificará la sección crítica y aplicará el mecanismo correcto para eliminarla. Se utilizará el módulo threading de Python, disponible en la instalación estándar del intérprete en Linux y Windows.

Entorno requerido: Python 3.8+ · Módulo threading (incluido en stdlib) · Editor de texto o IDE a elección (VS Code, Jupyter Lab). No se requieren instalaciones adicionales.

2.2 Parte A - Reproducción de la Race Condition

Cree el siguiente archivo Python y ejecútelo al menos 5 veces. Registre los resultados obtenidos.

```
Python
# race_condition.py
import threading

contador = 0
ITERACIONES = 100_000

def incrementar():
    global contador
    for _ in range(ITERACIONES):
        contador += 1 # <-- operación NO atómica

def decrementar():
    global contador
    for _ in range(ITERACIONES):
        contador -= 1 # <-- operación NO atómica

t1 = threading.Thread(target=incrementar)
t2 = threading.Thread(target=decrementar)

t1.start()
t2.start()
t1.join()
t2.join()

print(f'Resultado final: {contador} (esperado: 0)')
```

Registro de ejecuciones — Anote el valor obtenido en cada ejecución:

Ejecución 1	Resultado	Ejecución 2	Resultado	Ejecución 3	Resultado
Intento 1		Intento 2		Intento 3	
Intento 4		Intento 5		¿Hubo 0?	Sí / No

Preguntas de análisis

- ¿Por qué el resultado casi nunca es 0, si matemáticamente incrementar y decrementar 100.000 veces debería cancelarse?
- Identifique y marque con un comentario en el código la sección crítica. Explique por qué la operación contadora ± 1 no es atómica a nivel de instrucciones de máquina (ayuda: piense en las 3 instrucciones read-modify-write).

2.3 Parte B - Solución con Lock (Mutex)

Modifique el programa anterior para eliminar la Race Condition utilizando `threading.Lock()`. Complete el código donde se indica y ejecútelo 5 veces.

```
Python
# mutex_solution.py
import threading

contador = 0
ITERACIONES = 100_000
lock = threading.Lock() # <-- mutex creado

def incrementar():
    global contador
    for _ in range(ITERACIONES):
        # TODO: adquirir el lock aquí
        _____
        contador += 1
        # TODO: liberar el lock aquí
        _____

def decrementar():
    global contador
    for _ in range(ITERACIONES):
        # TODO: complete siguiendo el mismo patrón
        _____
        contador -= 1
        _____

t1 = threading.Thread(target=incrementar)
t2 = threading.Thread(target=decrementar)
t1.start(); t2.start()
t1.join(); t2.join()
print(f'Resultado final: {contador} (debe ser 0)')
```

Registro con mutex — Complete la tabla:

Métrica	Sin mutex (Parte A)	Con mutex (Parte B)
¿Resultado siempre 0?		
Tiempo de ejecución aprox.		
¿Resultado determinístico?		

Preguntas de análisis B

- La versión con mutex es más lenta que la versión sin mutex. ¿A qué se debe ese overhead? ¿Cómo se relaciona esto con el concepto de cambio de contexto estudiado en planificación?
- Usando el patrón [sección de entrada | sección crítica | sección de salida | sección restante], identifique cada parte en el código de la función incrementar() con mutex.

2.4 Parte C - Solución con Semáforo

Python también ofrece `threading.Semaphore()`. Reimplemente la solución del contador utilizando un semáforo binario (valor inicial = 1) en lugar del Lock y verifique que los tres requisitos de la sección crítica se satisfacen.

```
Python
# semaphore_solution.py
import threading

contador = 0
ITERACIONES = 100_000

# TODO: Cree un semáforo binario (valor inicial = 1)
sem = _____

def incrementar():
    global contador
    for _ in range(ITERACIONES):
        # TODO: operación wait (P)
        _____
        contador += 1
        # TODO: operación signal (V)
        _____

# TODO: implemente decrementar() con el mismo patrón

# TODO: cree, inicie y espere los hilos t1 y t2
# ...
print(f'Resultado: {contador}')
```

Pregunta C1

¿Cuál es la diferencia conceptual entre `threading.Lock()` y `threading.Semaphore(1)` en Python? ¿Son equivalentes en todos los casos? Fundamente con los requisitos de mutua exclusión, progreso y espera acotada.

Actividad 3 – Productor-Consumidor con Semáforos

3.1 Contexto

El problema Productor-Consumidor (Bounded Buffer) es el modelo de referencia para la coordinación entre procesos que comparten un recurso de capacidad limitada. En esta actividad implementará la solución clásica con tres semáforos, tal como la describe Silberschatz et al. (2018, Cap. 6).

Descripción del problema: Un hilo productor genera elementos y los deposita en un buffer circular de capacidad N. Un hilo consumidor extrae elementos del buffer. Si el buffer está lleno, el productor debe esperar. Si está vacío, el consumidor debe esperar.

3.2 Diseño previo (en papel, 10 min)

Antes de escribir código, responda el siguiente diseño conceptual en grupo:

Pregunta de diseño 1

¿Cuántos semáforos son necesarios para esta solución? Nombre cada uno, indique su valor inicial y su propósito específico. Relacione cada semáforo con uno de los tres requisitos de la sección crítica.

Semáforo	Valor inicial	Tipo	Propósito
mutex	1	Binario	Acceso exclusivo al buffer
empty			
full			

Pregunta de diseño 2

Escriba en pseudocódigo el esquema de las funciones producir() y consumir(), indicando el orden EXACTO de las operaciones wait/signal. ¿Por qué es crítico el orden? ¿Qué ocurre si se invierte?

Pseudocódigo

Pseudocódigo – Productor

```
def producir():
    item = generar_item()
    wait( _____ ) # 1. verificar espacio
    wait( _____ ) # 2. exclusión mutua
    buffer.agregar(item)
    signal( _____ ) # 3. liberar exclusión
    signal( _____ ) # 4. avisar elemento disponible
```

Pseudocódigo – Consumidor

```
def consumir():
    wait( _____ ) # 1. verificar elemento
    wait( _____ ) # 2. exclusión mutua
    item = buffer.extraer()
    signal( _____ ) # 3. liberar exclusión
    signal( _____ ) # 4. avisar espacio disponible
    procesar(item)
```

3.3 Implementación en Python

Implemente la solución completa con base en el diseño de la sección anterior. El código base está parcialmente completado. Rellene los espacios marcados con TODO.

```
Python
# productor_consumidor.py
import threading, time, random

N = 5                                # capacidad del buffer
buffer = []
mutex = threading.Semaphore(1)
empty = threading.Semaphore(N) # TODO: valor inicial correcto?
full = threading.Semaphore(0) # TODO: valor inicial correcto?
ITEMS = 10                          # items totales a producir

def producir():
    for i in range(ITEMS):
        item = random.randint(1, 100)
        time.sleep(random.uniform(0.05, 0.2))
        # TODO: semáforo correcto antes de mutex
        _____.acquire()
        mutex.acquire()
        buffer.append(item)
        print(f'[P] Produjo {item:3d} | Buffer: {buffer}')
        mutex.release()
        # TODO: semáforo correcto después de mutex
        _____.release()

def consumir():
    for i in range(ITEMS):
        time.sleep(random.uniform(0.1, 0.3))
        # TODO: complete acquire/release en orden correcto
        _____.acquire()
        _____.acquire()
        item = buffer.pop(0)
        print(f'[C] Consumió {item:3d} | Buffer: {buffer}')
        _____.release()
        _____.release()

p = threading.Thread(target=producir)
c = threading.Thread(target=consumir)
p.start(); c.start()
p.join(); c.join()
print('Fin. Buffer final:', buffer)
```

3.4 Preguntas de análisis

- Ejecute el programa con N=1 (buffer de capacidad 1). Describa el comportamiento observado. ¿Cuál es el patrón de ejecución alternante entre productor y consumidor?
- Elimine el semáforo mutex (solo el acquire/release de mutex, mantenga empty y full) y ejecute con N=5. ¿Qué problema puede ocurrir ahora? Registre si observó algún estado inconsistente del buffer.
- Agregue un segundo consumidor (c2) al programa. ¿Funciona correctamente? ¿Qué garantiza que dos consumidores no extraigan el mismo elemento simultáneamente? Relacione su respuesta con el requisito de mutua exclusión.

4. Criterios de evaluación

Rúbrica detallada

Criterio	Logrado (20 pts)	Satisfactorio (15 pts)	Insuficiente (5 pts)	No presentado (0)
A1 - Diagrama de Gantt FCFS correcto	Sin errores. Todos los tiempos correctos.	1–2 errores menores de timing.	Errores conceptuales en orden o inicio.	No realizado.
A1 - Diagrama de Gantt RR $q=3$ correcto	Turnos correctos, expulsiones bien marcadas.	Error en 1–2 turnos o quantum.	Confunde apropiativo con no apropiativo.	No realizado.
A1 - Cálculo de métricas	Espera promedio exacto en ambas políticas.	Fórmula correcta, error aritmético menor.	Fórmula incorrecta o datos incompletos.	No presentado.
A1 - Análisis comparativo	Justificación con criterios de Silberschatz.	Respuesta parcial o sin bibliografía.	Respuesta genérica sin fundamento.	No respondido.
A2 - Reproducción de Race Condition	5 ejecuciones, resultados variados documentados.	3–4 ejecuciones o registro incompleto.	1–2 ejecuciones sin análisis.	No ejecutado.
A2 - Implementación mutex	Código correcto, resultado siempre 0.	Código funcional con error menor de lógica.	Lock mal colocado, resultado incorrecto.	No implementado.
A2 - Implementación semáforo	Semáforo binario correcto, equivalencia explicada.	Funcional pero sin análisis de equivalencia.	Valor inicial incorrecto o semáforo mal usado.	No implementado.
A2 - Análisis SC (3 requisitos)	Los 3 requisitos identificados en el código.	2 de 3 requisitos identificados.	Solo mutua exclusión mencionada.	No respondido.
A3 - Diseño de semáforos en tabla	Tres semáforos correctos con valores y propósito.	2 de 3 semáforos correctamente definidos.	Solo mutex identificado.	No presentado.
A3 - Pseudocódigo con orden correcto	Orden wait/signal exactamente correcto.	Orden con un error no crítico.	Orden invertido en alguna operación crítica.	No presentado.
A3 - Implementación Python funcional	Buffer nunca excede N, resultado correcto.	Funciona con N grande, falla con $N=1$.	Deadlock o error de ejecución.	No implementado.
A3 - Análisis de experimentos	Observaciones correctas y bien fundamentadas.	Observaciones parciales o sin fundamento.	Respuesta genérica sin relación al código.	No respondido.

6. Referencias y Recursos de Apoyo

- Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson. Cap. 7.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley. Cap. 18.